

Market Success Viability Predictor

An explainable, aspect-based ML system for pre-launch product viability

A capstone project that mines real consumer reviews at the aspect level, learns which aspect patterns drive market success per category, and lets a founder describe an unlaunched product to receive an explainable Market Viability Assessment grounded in real data.

Track	ML (LLM extraction, XGBoost, SHAP, bridging, web app)
Domains	5 physical hardware categories
Live app	React + FastAPI, deployed on Render (Docker)
Document date	September 2026

Read this document top to bottom: it explains the whole project in the order it actually works — from the idea, through data and the ML pipeline, to the app, deployment, and honest limitations.

1. What the project is

The system answers one question: **if a founder is about to launch a new hardware product, how would consumers likely receive it — and why?** It does not forecast revenue or units sold. Instead it produces a **Market Viability Assessment**: a 0–100 viability score plus a transparent, aspect-by-aspect explanation of what lifts or drags that score.

The core insight is that thousands of existing product reviews encode what consumers in a category reward and punish. If we extract that knowledge at the level of concrete *aspects* (build quality, value for money, reliability, and so on), we can judge a brand-new product's specs against those learned expectations.

Three components each do what they are best at:

- **LLM (perception translator)** — turns unstructured review text, and later a new product's specs, into structured 0–10 aspect scores.
- **XGBoost (outcome predictor)** — learns, from thousands of real products, which aspect patterns correlate with market success in each category.
- **SHAP (decision support)** — decomposes each prediction into the aspects raising and lowering it, so the answer is explainable, not a black box.

2. The core idea and an honest reframe

Early on, the project was framed as a 'success predictor'. That framing invited a fair critique: you cannot predict a product's own reviews from its own reviews. The design was therefore reframed as a **category-level viability assessment**:

- The model **trains on existing products** in a category and learns the relationship between aspect profiles and a success proxy.
- At prediction time, a **new** product's specs are translated into the same aspect space (the 'bridging' step) and judged against the learned category patterns.

This resolves the temporal problem — we train on the past and estimate for an unreleased product — and it sets honest expectations: the label is a review-derived *proxy* for success, not real sales.

3. Data foundation

Two real, free, reliable sources underpin the system:

- **Amazon Reviews 2023 (McAuley Lab, HuggingFace)** — the physical-product backbone; streamed and filtered.
- **Google Play** — originally an app track (later removed; see below).

Category cleanup — a real data-quality fix

An audit of the raw categories exposed heavy contamination: 'smartphones' was ~64% accessories (cases, chargers, gimbals, signal boosters), and 'kitchen appliances' was ~76% consumables (coffee filters, pods, replacement parts). These polluted price statistics, aspect baselines, and the trained model itself. The root cause was over-broad keyword filters (e.g. matching any title containing 'iphone'). In response:

- **Smartphones and kitchen appliances were dropped** (only ~29 real phones survived filtering — too thin).

- **A clean 'ice makers' category was built** from the Appliances pool with a strict machine-only filter, then LLM-extracted — replacing the contaminated kitchen category.
- **The app track was removed entirely** (its aspects-only model failed at $R^2 \approx 0.10$, and it broadened scope without adding signal), making the system physical-only and coherent.

The final, validated dataset (physical only):

Category	Products	Reviews
Wireless headphones	138	16,473
Bluetooth speakers	136	12,225
Ice makers (new)	147	6,988
Smartwatches	116	3,981
Power banks	103	3,932
Total	640	43,599

Data-quality safeguards baked in: verified-purchase filtering, statistical fake-review flagging (review-burst, extreme rating-J, near-duplicate text), an English/length cleaning gauntlet, and per-aspect coverage tracking so unmentioned aspects are never invented.

4. The pipeline, stage by stage

One command takes clean data to trained models. Each stage writes to disk and is independently re-runnable, so fixing a later stage never re-runs expensive earlier work.

A · Load & verify contract

Reads the data manifest, refuses to run unless validation passed, and loads everything into typed structures with clear errors.

B · LLM aspect extraction

Each review is turned into 0–10 scores for 9 physical aspects using a **null-forcing prompt**: an aspect is scored only if the review explicitly discusses it, otherwise it returns null with a verbatim evidence phrase. A multi-provider pool (Gemini, Groq, Cerebras, Mistral, SambaNova) with rate-limiting and resume ran ~44k extractions crash-safe.

C · Label engineering

Computes the success proxy per product: a within-category, min-max normalized blend of review volume (40%), average rating (35%), and review velocity (25%). Fake-review-flagged products are excluded (9 excluded). A per-category std check guards against flat, unlearnable labels.

D · Feature matrix

Aggregates per-review aspect scores up to per-product means plus mention-rates, joins the label, and writes a flat numeric matrix. The exact ordered column list is saved to disk as the manifest that prediction reuses — training and prediction can never misalign. Missing aspects stay NaN (XGBoost handles it natively; no imputation).

E · XGBoost training + SHAP

Trains two variants per model with 5-fold CV: a **full** model (all features) and an **aspects-only** model (drops price/rating/velocity) — the honest measure of what the LLM features contribute. SHAP summaries and a full metrics report are saved.

F · Category profiles

Aggregates each category into the intelligence the bridging step needs: price distribution, average aspect scores, pain points and strengths with real evidence phrases, and representative products.

G · Bridging + prediction service

The live path: a new product's specs + the category profile go to the bridging LLM, which estimates aspect scores in the trained feature space; XGBoost scores them; SHAP explains them.

5. The bridging layer — the novel piece

The hardest and most original component is **bridging**: translating a product that has no reviews yet into the same aspect space the model was trained on.

When a founder enters specs (e.g. a 10,000 mAh power bank at \$24 with USB-C PD), the system fetches that category's profile and asks the LLM to reason spec-by-spec against real category facts: *'priced below the median with fast charging → high value_for_money; sealed body in a category where repairability is a known pain point → low repairability'*. The LLM returns a full aspect vector with one-sentence reasoning per aspect, which flows into XGBoost and SHAP.

Why not let the LLM predict success directly? Because the LLM has no statistical grounding for how much each aspect matters in a specific category, cannot capture non-linear aspect interactions, and cannot be decomposed by SHAP. The division of labour is the point: the LLM understands language, XGBoost does the statistics, SHAP explains.

6. Model results (reported honestly)

Model variant	Cross-validated R ²	Meaning
Full model	0.888 (PASS)	Includes rating/price/velocity — inflated by rating being 35% of the label
Aspects-only	0.566 (PASS)	Honest headline: what the LLM aspect features contribute alone
External holdout	p = 0.317, AUC = 0.743	59 held-out products — modest but real ranking signal

Per-category (full / aspects-only) R²: bluetooth_speakers 0.92 / 0.57, power_banks 0.87 / 0.53, smartwatches 0.85 / 0.47, ice_makers 0.82 / 0.32, wireless_headphones 0.80 / 0.33. Trained on 631 products (9 fake-review-flagged excluded). The reframed dataset also lowered the rating–label leakage correlation to ~0.46.

The project deliberately leads with the **aspects-only** number and the holdout, not the flashy 0.888 — the honest story is that the LLM aspect features carry genuine, category-specific signal, while acknowledging the proxy label limits absolute precision.

7. The application

A polished React + FastAPI single-page app. FastAPI serves the built React UI and a small JSON API; the predictor runs in-process. Four areas:

- **Simulator** — enter category, price, and a spec description; get the viability range, market-percentile rank, bridging confidence, a viability donut, an aspect-vs-category bar chart, a design radar, a market-positioning grid of real category products, SHAP risk/strength cards, and a full explainability path map.
- **Spec Coach** — critiques a draft spec, flags which of the 9 aspects lack concrete evidence, and asks sharpening questions.
- **Analyzer** — surfaces the model's own cross-validation metrics.
- **Explorer** — browse category intelligence.

Trust and honesty safeguards

Because the estimate is only as good as the input spec, the app makes reliability visible:

- **Grounding-adjusted confidence** — the LLM's self-confidence is scaled by the fraction of aspects actually backed by evidence, so vague or nonsense specs can no longer report high confidence.
- **Below 0.40** — a clear amber 'low-confidence, indicative only' banner and a tinted confidence card.
- **Below 0.20** — the prediction is **withheld entirely** and replaced with a 'not enough detail to predict' explainer, rather than showing a meaningless number.
- **SHAP cards** are aspect-level and labelled by score direction ('raising / lowering your viability'), avoiding the earlier confusion where a low aspect appeared as a 'strength'.

8. Deployment

The app is containerized (Dockerfile) and deployed as a free web service on **Render**, serving the FastAPI app directly on one port. API keys are supplied as environment-variable secrets and never committed; the public GitHub repository was built from a clean, single-commit history and scanned to guarantee no keys are exposed.

Two hosts were ruled out along the way, which is itself a useful finding: **Streamlit Cloud** cannot serve this React+FastAPI app because it only exposes its own server (custom API routes are unreachable), and **Hugging Face Spaces** now charges for the Docker runtime this app needs. Render's free Docker tier keeps the premium UI intact.

9. Known limitations and independent audit

An independent automated audit stress-tested the live system against 25 real historical products. It confirmed the limitations already disclosed and surfaced two genuine flaws worth stating plainly:

- **Proxy label & compressed range** — success is a review-volume/rating/velocity proxy, not sales. Scores cluster in a narrow band per category, so the model ranks better than it separates blockbusters from average products.
- **Inverted price response** — because the headline model excludes price directly and the LLM infers 'premium' from a high price, viability can rise with price instead of falling. There is no demand curve in the system.

- **Specification-inversion paradox** — advanced proprietary features (e.g. ECG, high-end ANC) are hard for the LLM to map onto 9 generic consumer aspects, so stripping them can occasionally raise the score.
- **Out of scope by design** — the system models consumer-sentiment aspects only. It has no brand-equity, retail-distribution, ecosystem-moat, or manufacturing-margin features and never claimed to; those belong to a different, commercial-forecasting product.

These are treated as explicit threats-to-validity rather than hidden. The confidence and abstain safeguards above are the system's honest response to thin or ambiguous inputs, and the aspects-only + holdout reporting keeps the headline claims defensible.

10. How to run

Rebuild the ML pipeline end-to-end (from cleaned data to trained models and profiles):

```
py src/training/run_training.py --force
py src/prediction/category_profiler.py --force
```

Run the app locally (React + FastAPI on <http://127.0.0.1:8000>):

```
py src/app/app.py
```

Summary: a category-scoped, aspect-based viability tool built on a clean two-source dataset, an LLM→XGBoost→SHAP pipeline, and a deployed explainable web app — reported with deliberate honesty about what its proxy label can and cannot do.